Median Of Two Sorted Arrays

AlgoExpert · Very Hard · Searching

The problem

- Median of the union of two sorted arrays via binary-searched partition.

Intuition

- Binary search a cut in the smaller array so the left halves of both hold exactly half the elements.

Brute force vs optimal

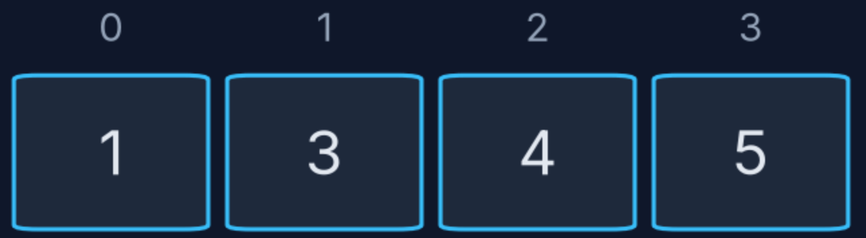
- Merge then pick middle → O(n + m)
- Binary search partition in the shorter array → O(log min(n, m))

Approach

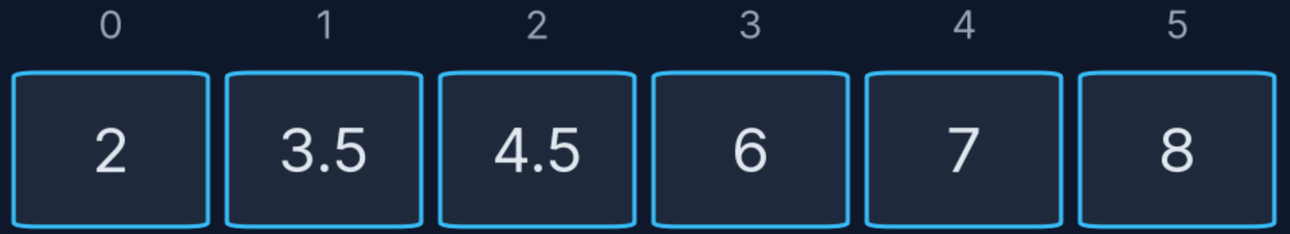
- 1. Search the cut in the smaller array
- 2. Derive the cut in the other
- 3. Check that the edges are in order
- 4. Median from the edge values

Worked example

a (shorter)



b



Partition both so the left halves have half the elements

Worked example

a



b

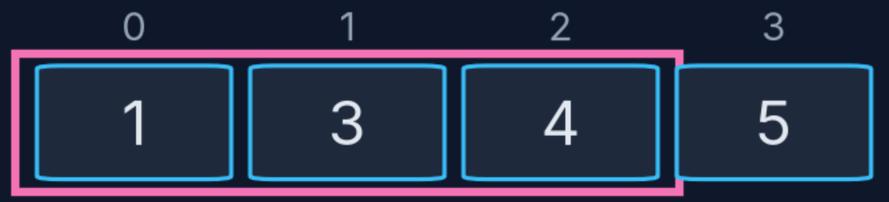



```
i=2, j=3: aL=3 ≤ bR=6, bL=4.5 > aR=4 → move right
```

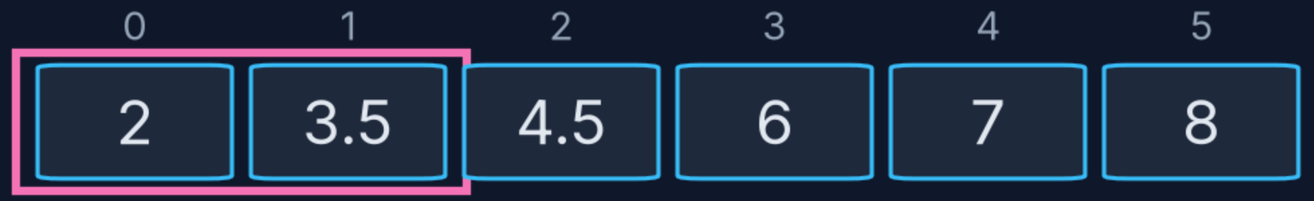
Binary search the cut in the shorter array

Worked example

a



b




```
i=3, j=2: aL=4 ≤ 4.5, bL=3.5 ≤ 5 ✓
even total → (max(4, 3.5) + min(5, 4.5)) / 2 = 4.25
```

Valid cut → median 4.25

Java solution

```java
public class MedianOfTwoSortedArrays {
    public static float medianOfTwoSortedArrays(int[] a, int[] b) {
        if (a.length > b.length) return medianOfTwoSortedArrays(b, a);
        int n = a.length, m = b.length, half = (n + m + 1) / 2, lo = 0, hi = n;
        while (true) {
            int i = (lo + hi) / 2, j = half - i;
            int aL = i == 0 ? Integer.MIN_VALUE : a[i - 1], aR = i == n ? Integer.MAX_VALUE : a[i];
            int bL = j == 0 ? Integer.MIN_VALUE : b[j - 1], bR = j == m ? Integer.MAX_VALUE : b[j];
            if (aL > bR) hi = i - 1;
            else if (bL > aR) lo = i + 1;
            else {
                if ((n + m) % 2 == 1) return Math.max(aL, bL);
                return (Math.max(aL, bL) + (float) Math.min(aR, bR)) / 2;
            }
        }
    }
}
```

Complexity

- Time: O(log(min(n, m)))
- Space: O(1)

Edge cases & pitfalls

- Search the shorter array
- Sentinels at the edges
- Odd vs even totals

Interview tips & follow-ups

- Explain the partition condition

Recap

- Binary search a cut in the smaller array so the left halves of both hold exactly half the elements.
- Complexity: O(log(min(n, m)))